

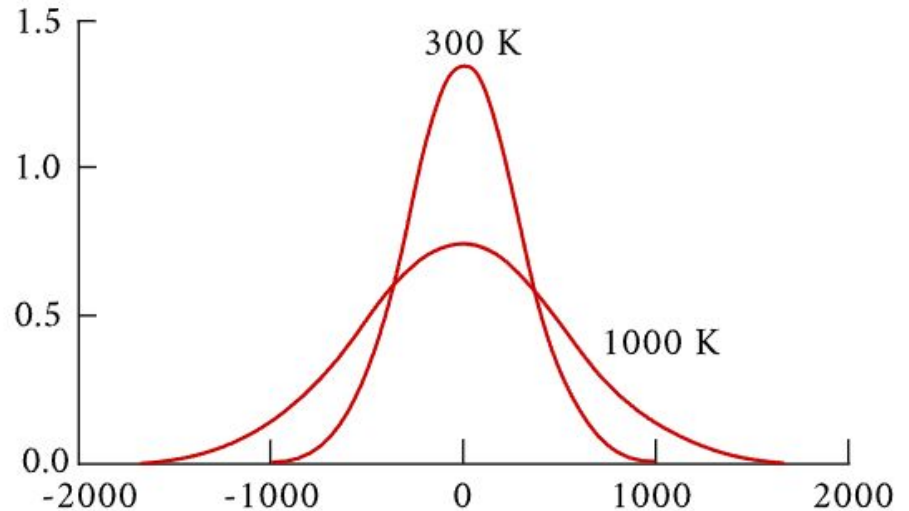


# Writing our own code for periodic LJ system and benchmark it against LAMMPS

Adit Rambhia, Nishant Tatar, Vihaan Jhaveri

## Defining the system

- Argon gas, of 500 atoms
- $T = 1000$  K
- Micro canonical: NVE ensemble
- Timestep = 1 fs
- Velocity: Gaussian Distribution
- Box Size = 200 Å





```
# Simulation parameters
n = 500
Edgelenh_box = np.float64(2e-8)
runtime_iterations = 100000
time_step = np.float64(1e-15)
Temp = np.float64(1000)

# Constants
k_b = np.float64(scipy.constants.k)
m_Ar = np.float64(6.642160e-26)

# Sigma and Epsilon for Argon
sigma = np.float64(3.4e-10)
epsilon = np.float64(1.65e-21)
cutoff_distance = np.float64(1e-9)
```

# Functions in Python Script

---

```
# Initialization functions
def init_pos(n, Edglength_box):
    return np.random.uniform(0, Edglength_box, (n, 3))

def init_vel(n, Temp):
    vel = np.random.normal(0, np.sqrt(k_b * Temp / m_Ar), (n, 3))
    return vel - np.mean(vel, axis=0)

# Minimum image distance with broadcasting
def minimum_image_distance(r, Edglength_box):
    delta_r = r[:, np.newaxis, :] - r[np.newaxis, :, :]
    return delta_r - Edglength_box * np.round(delta_r / Edglength_box)
```

```
# Lennard-Jones potential and force calculations
def lj_potential_and_force(r_ij, cutoff_distance):
    r_sq = np.sum(r_ij ** 2, axis=2)
    mask = (r_sq < cutoff_distance ** 2) & (r_sq > 0)

    r_6 = sigma_6 / r_sq[mask] ** 3
    r_12 = r_6 ** 2
    U = np.zeros_like(r_sq)
    F_mag = np.zeros_like(r_sq)

    # Lennard-Jones potential and force only for particles within cutoff
    U[mask] = 4 * epsilon * (r_12 - r_6)
    F_mag[mask] = (epsilon_48 * r_12 - epsilon_24 * r_6) / np.sqrt(r_sq[mask])

    return U.sum() / 2, F_mag[:, :, np.newaxis] * r_ij
```



```
# Kinetic Energy
def kinetic_energy(v):
    return 0.5 * m_Ar * np.sum(v ** 2)

# Temperature Calculation
def temperature(v):
    return 2 * kinetic_energy(v) / (3 * k_b * n)

# Function to write atom positions in .xyz format
def write_xyz(filename, r, iteration):
    with open(filename, 'a') as f:
        f.write(f"ITEM: TIMESTEP \n{iteration}\n")
        f.write(f"ITEM: NUMBER OF ATOMS\n{n}\n")
        f.write("ITEM: BOX BOUNDS pp pp pp\n")
        f.write(f"0 {Edgelenh_box*1e+10}\n0 {Edgelenh_box*1e+10}\n0 {Edgelenh_box*1e+10}\n")
        f.write("ITEM: ATOMS id type x y z\n")
        for i in range(n):
            f.write(f"{i+1} 1 {np.round(r[i, 0]*1e+10,3)} {np.round(r[i, 1]*1e+10,3)} {np.round(r[i, 2]*1e+10,3)}\n")
```



```
# Main Loop with Velocity Verlet
def main_loop(n, Edgelenlength_box, runtime_iterations, time_step, Temp):
    r = init_pos(n, Edgelenlength_box)
    v = init_vel(n, Temp)

    # Initial force calculation
    r_ij = minimum_image_distance(r, Edgelenlength_box)
    _, F = lj_potential_and_force(r_ij, cutoff_distance)
    F_sum = F.sum(axis=1)

    print("Iteration", "Potential Energy", "Kinetic Energy", "Temperature")
```

```

for iteration in range(runtime_iterations):
    # Position update
    r += v * time_step + 0.5 * (F_sum / m_Ar) * time_step ** 2
    r = r % Edgelenh_box

    if iteration % 1000 == 0:
        current_temperature = temperature(v)
        v = v * np.sqrt(Temp / current_temperature)

    # New force calculation
    r_ij = minimum_image_distance(r, Edgelenh_box)
    U, F_new = lj_potential_and_force(r_ij, cutoff_distance)
    F_sum_new = F_new.sum(axis=1)

    # Velocity and force update
    v += 0.5 * (F_sum + F_sum_new) / m_Ar * time_step
    F_sum = F_sum_new

    if iteration % 1000 == 0:
        K = kinetic_energy(v)
        T = temperature(v)
        print(iteration, U, K, T)
        write_xyz(output_filename, r, iteration)

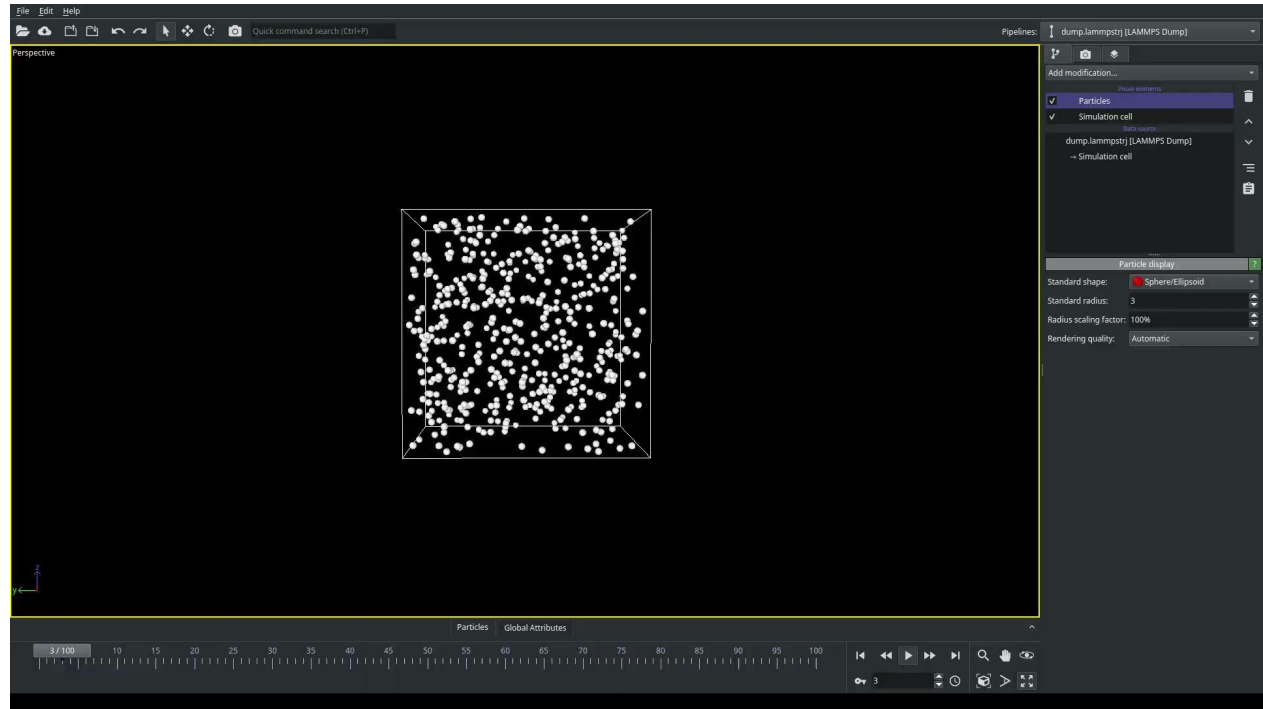
return U, K, T

```

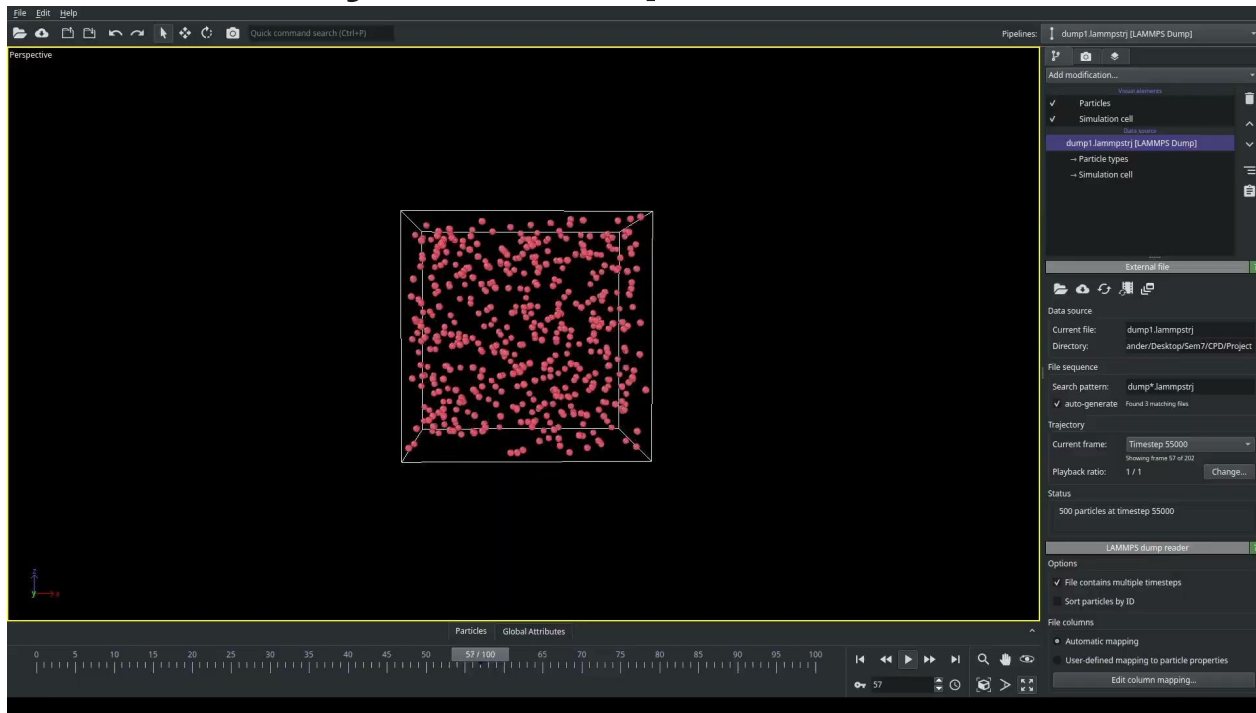


# Simulation of LAMMPS script

- LJ parameters in real units  
 $\sigma = 3.405, \epsilon = 0.238$
- PBC, with box length = 200 Å



# Simulation of Python Script

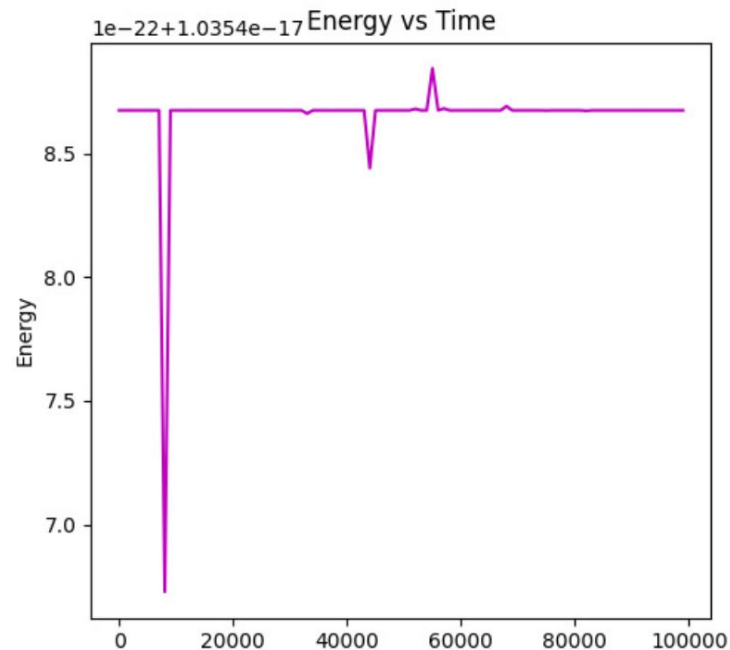
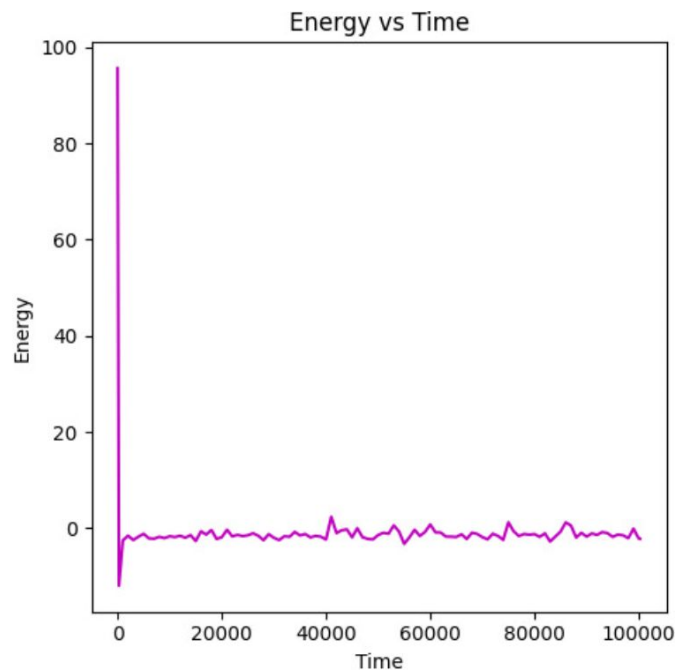




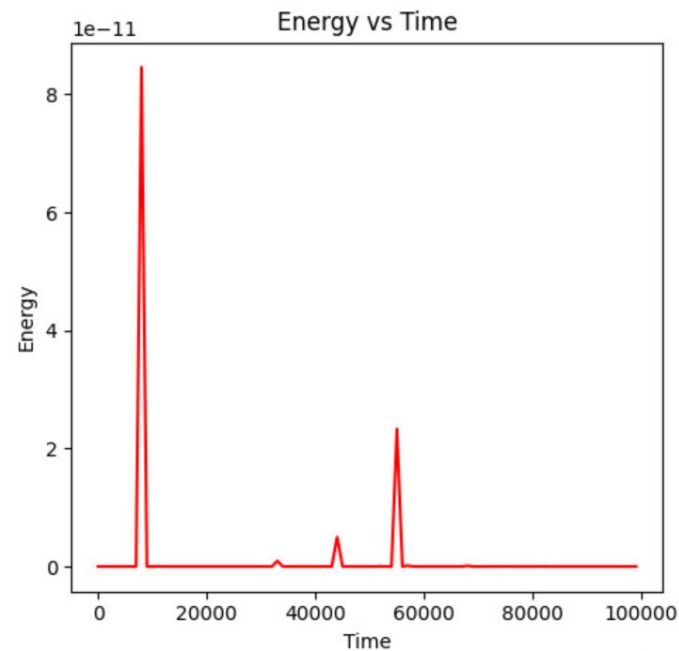
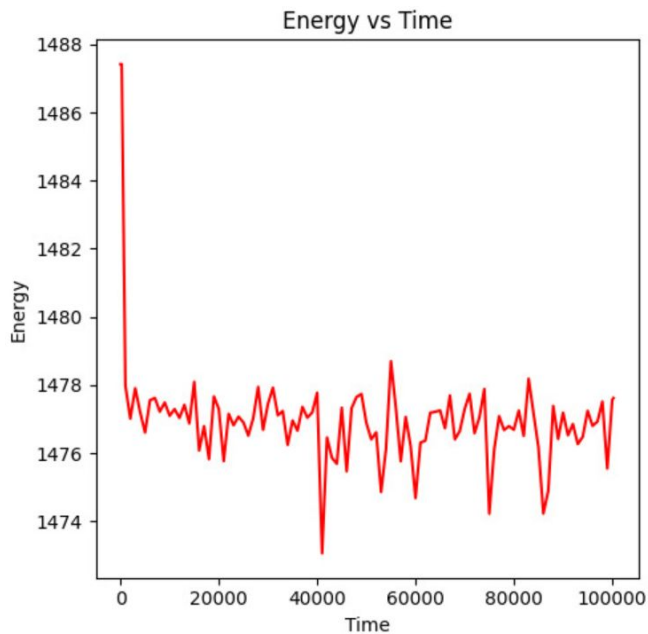
## Benchmarking with LAMMPS:

- Time for simulation to run: LAMMPS: 1.69 s , Python: 1695.722 s
- Std deviation for Kinetic energy: LAMMPS : 0.09 % mean , Python: 0.00% of mean
- Std deviation for Potential energy: LAMMPS : -92.03 % mean , Python: 762% of mean

# Benchmarking with LAMMPS: Potential Energy



# Benchmarking with LAMMPS: Kinetic Energy





**THANK YOU!**